Informe del taller 4 - Monitoreo con APM y prueba de estrés

Optativa IV - Sistemas de Tiempo Real Distribuidos

Fundación Escuela Tecnológica de Neiva (FET)

Medición: 26 de octubre de 2026, 10:10 a 10:30 (UTC-5)

1. Respuesta

Esta infraestructura soporta 72 peticiones por segundo con 10 usuarios concurrentes antes de incumplir los criterios declarados el 25 de octubre de 2026. El recurso que impone ese límite es la API: un proceso Node con tope de 1 CPU. A partir de 25 usuarios el caudal baja y la latencia p95 pasa de 500 ms. La memoria de la API no se llenó y el tiempo del SELECT no explica la espera.

2. Criterios, fijados antes de medir

Quedaron escritos el 25 de octubre de 2026 en docs/criterios_saturacion.md. No se cambiaron después de ver la corrida.

- Tasa de error menor o igual a 1 %.
- Latencia p95 menor o igual a 500 ms.
- El throughput tiene que seguir subiendo al aumentar los usuarios.

El punto de saturación es el último escalón que cumple los tres a la vez. El calentamiento de 1 minuto con 10 usuarios no entra en la tabla.

3. Cómo se midió

SigNoz recibe las trazas de la API por OpenTelemetry. k6 corre en otro contenedor y durante 3 minutos en cada escalón llama al CRUD de clientes: 70 % GET /clients, 20 % POST /clients y 10 % PUT /clients/:id, con datos distintos en cada escritura. Escalones: 10, 25, 50, 100, 200 y 400 usuarios, sin pausa entre peticiones.

Las peticiones por segundo, los percentiles y el error de la tabla salen de las trazas de servidor de SigNoz (GET /clients, POST /clients y PUT /clients/:id), contadas dentro de cada ventana de 3 minutos. En el escalón 1 el APM registró 12.955 peticiones; el contador de k6 en ese mismo tramo pasó de 11.082 a 24.081 iteraciones, 12.999 en 3 minutos, unas 72 por segundo. Coinciden.

La tabla que k6 imprime al final no sirve para la columna de peticiones por segundo. Su columna rate divide el conteo del escalón entre los 19,5 minutos de toda la prueba, y por eso muestra 11,10 en el escalón 1. Los percentiles y el porcentaje de error de esa tabla sí corresponden a cada escalón. El código 99 de k6 solo avisa de que se cruzaron los umbrales a partir del escalón 2.

4. Tabla de los seis escalones

Latencia y error: APM. Peticiones por segundo: conteo de trazas del APM dividido entre 180 segundos. CPU, memoria y conexiones: no se muestreó docker stats durante la corrida, y esas celdas no se rellenan con una estimación.

Esc	Usr	Pet/s	p50	p95	p99	Err%	CPU API	CPU BD	Mem	Conex
1	10	72,0	60	278	381	0,00	s/d	s/d	s/d	s/d
2	25	46,7	242	959	1158	0,00	s/d	s/d	s/d	s/d
3	50	38,1	938	2235	2626	0,00	s/d	s/d	s/d	s/d
4	100	31,2	2714	5505	6247	0,00	s/d	s/d	s/d	s/d
5	200	26,7	6796	13397	14439	0,00	s/d	s/d	s/d	s/d
6	400	24,4	15254	30208	31548	0,00	s/d	s/d	s/d	s/d

Los tiempos están en milisegundos y son los de la traza en el servidor. s/d significa sin dato de docker stats. En el escalón 6 k6 vio 1,37 % de fallos en el cliente (el p99 del cliente llegó a 60 s y k6 cortó esas peticiones). En SigNoz esas mismas peticiones que sí terminaron quedaron en HTTP 200 o 201, con 0 % de trazas en error. El criterio de error del APM se cumple en los seis escalones. El de latencia y el de crecimiento del caudal solo se cumplen en el 1.

Referencia del cliente, no usada como columna oficial de latencia: p95 de k6 338 ms, 1126 ms, 2455 ms, 5913 ms, 14101 ms y 31906 ms. Va por encima del servidor, como corresponde al viaje de ida y vuelta, y cruza 500 ms en el mismo escalón.

5. Punto de saturación

El último escalón que cumple los tres criterios es el 1: 10 usuarios y 72 peticiones por segundo. El p95 del APM es 278 ms, el error es 0 % y el caudal todavía está en su máximo. En el escalón 2, con 25 usuarios, el caudal baja a 47 peticiones por segundo y el p95 sube a 959 ms. Ahí ya fallan dos criterios.

6. Ficha del equipo

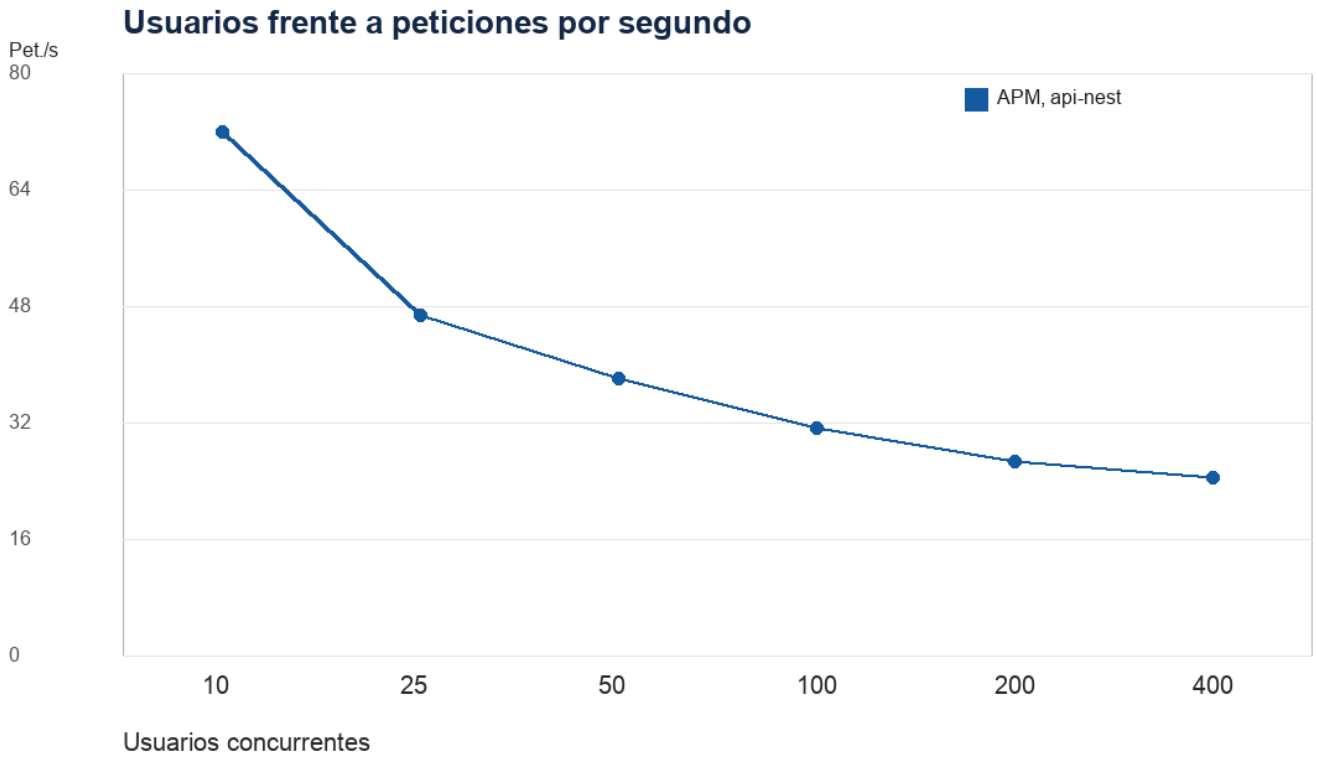
- Equipo: Apple M2 Max, 12 núcleos, 32 GB de RAM, macOS 26.7.1.
- Docker Desktop 23.0.5, con 4 CPU y 7,7 GB de RAM asignados a la máquina virtual. Los contenedores no ven los 12 núcleos ni los 32 GB.
- mysql: límite 1 CPU y 1 GB. max_connections = 151. Pool de la API: el de TypeORM, 10 conexiones.
- api (api-nest): límite 1 CPU y 512 MB.
- consumer: límite 0,5 CPU y 512 MB. No entra en esta prueba.
- rabbitmq: límite 0,5 CPU y 512 MB.
- signoz: límite 1 CPU y 1 GB. clickhouse: límite 1 CPU y 2 GB. otel-collector: límite 0,5 CPU y 512 MB.
- k6: límite 2 CPU y 1 GB, por encima de la API, para que el generador no sea el tope.

Lo que SigNoz sí registró del proceso Node durante la prueba: el heap usado no pasó de unos 68 MB, frente al límite de 512 MB del contenedor. La utilización del event loop llegó a 1,0 en todos los escalones y su retraso p99 pasó de 54 ms en el escalón 1 a 217 ms en el 6. Eso no es un porcentaje de CPU del contenedor; docker stats no se guardó.

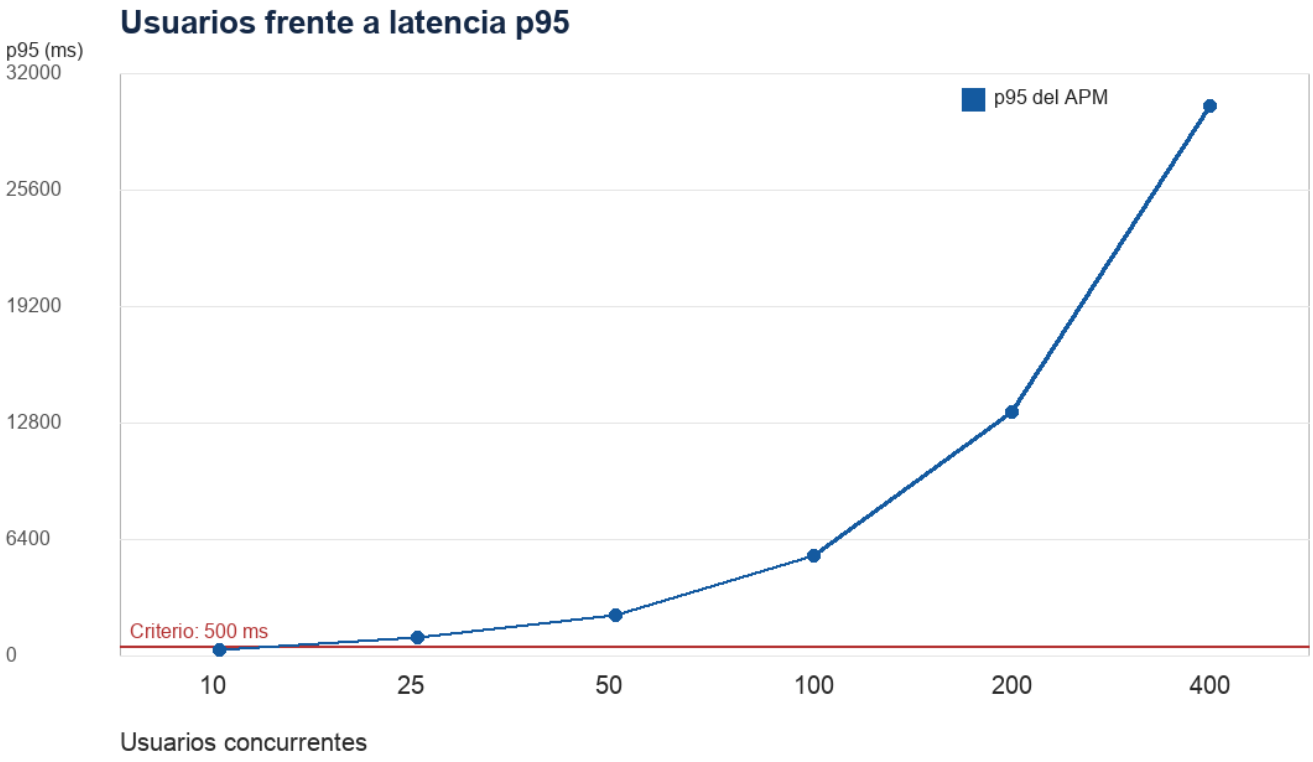
7. Gráficas

Las tres gráficas usan los usuarios de cada escalón y los valores de la tabla. La de error agrega la serie de k6 para mostrar el 1,37 % del escalón 6, que el APM no cuenta como error de servidor.

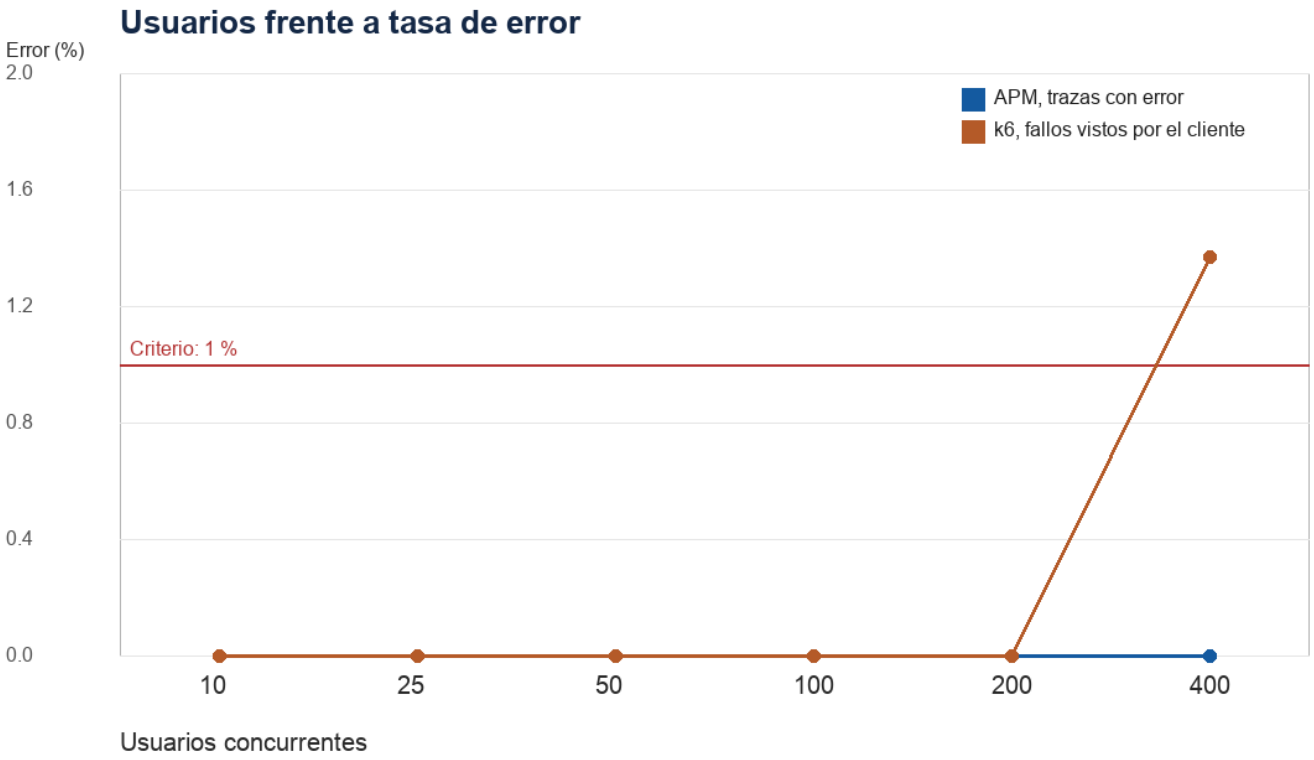
Usuarios frente a peticiones por segundo.



Usuarios frente a latencia p95. La línea roja es el criterio de 500 ms.



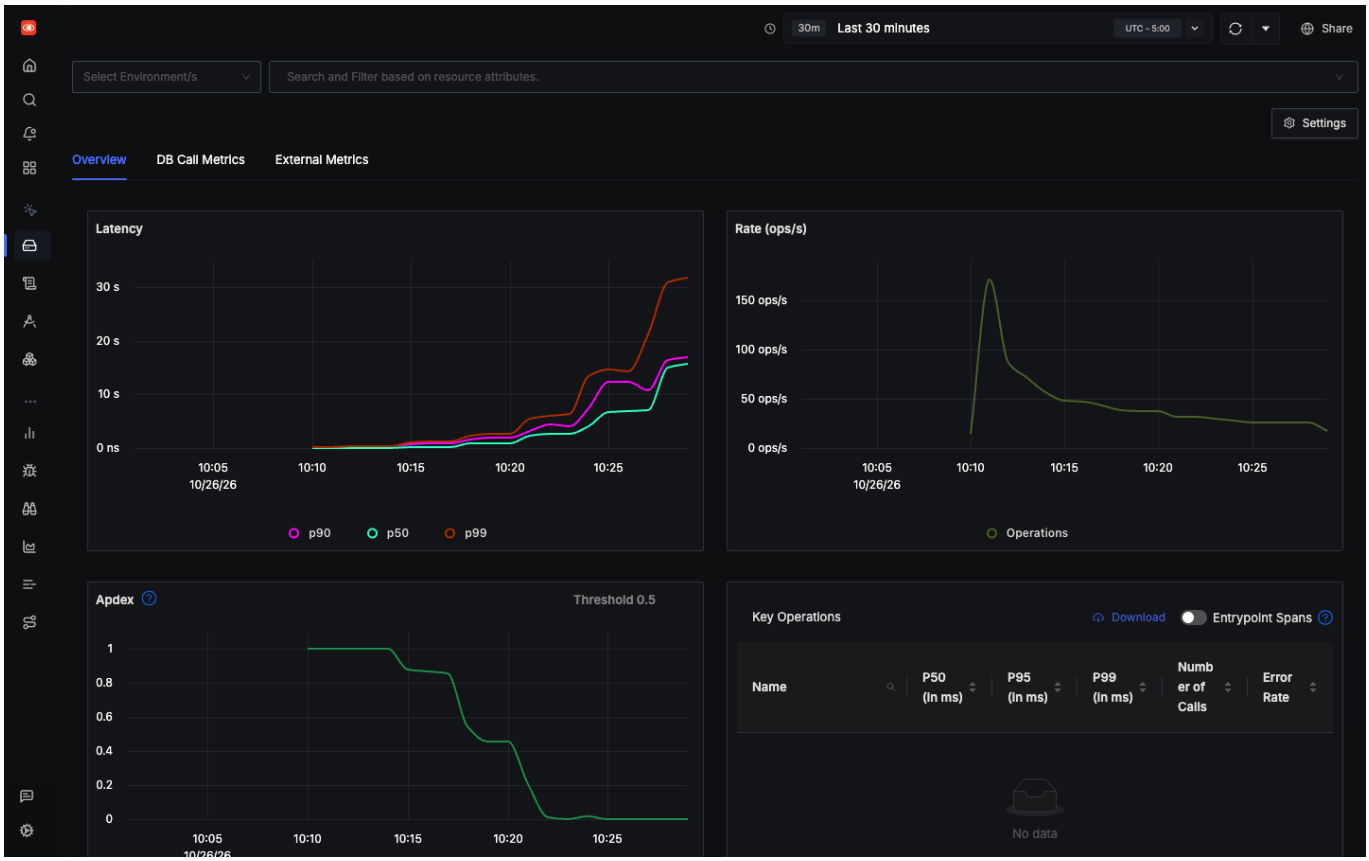
Usuarios frente a tasa de error. La línea roja es el criterio de 1 %.



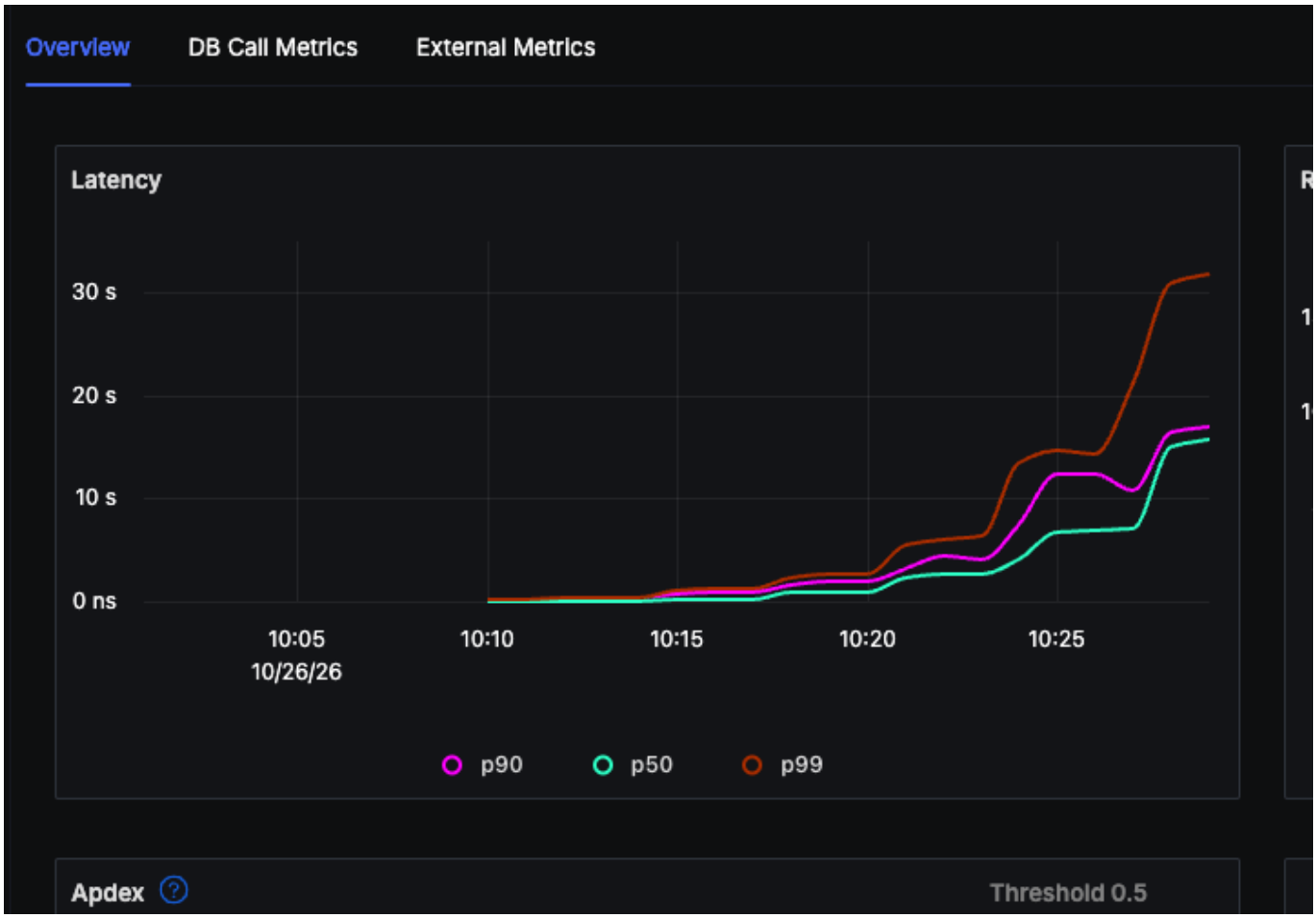
8. Capturas del APM

Panel del servicio api-nest, percentiles, tasa de error y una traza de GET /clients tomada al cierre de la prueba (26 de octubre de 2026, 10:30:28). Esa traza dura 14,91 s. El tramo findAll dura 14,89 s y el SELECT dura 406 ms.

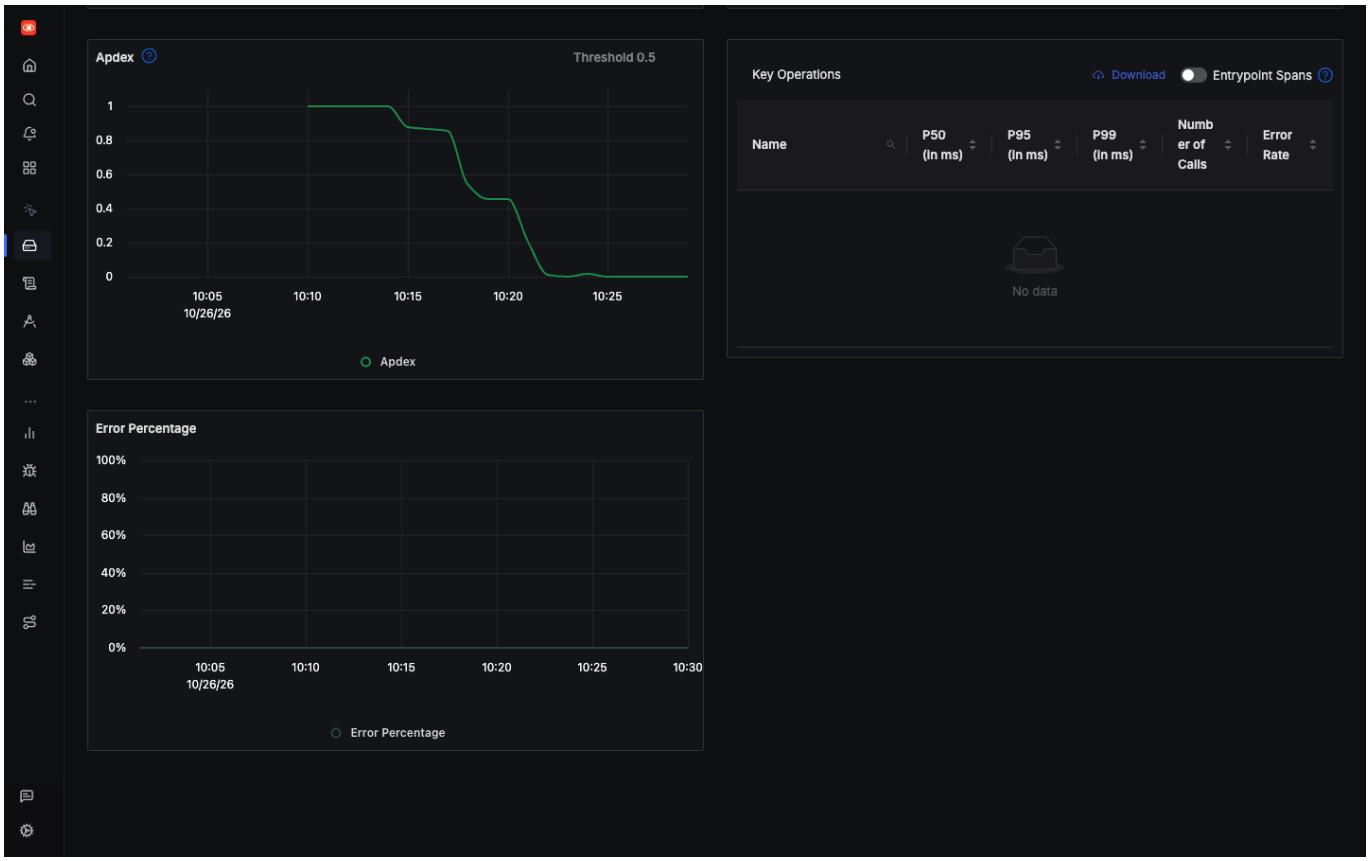
Panel del servicio api-nest durante la prueba.



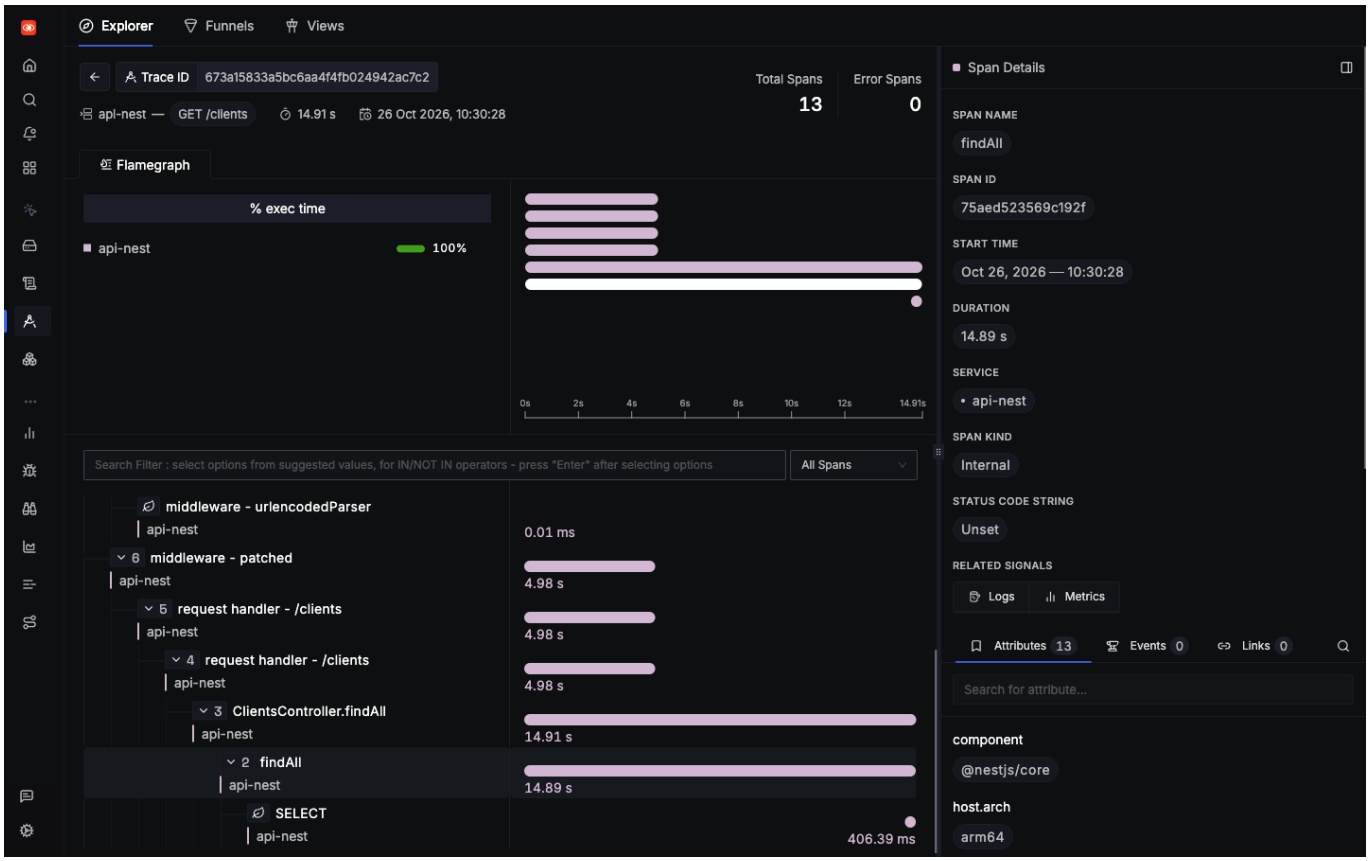
Latencia por percentiles (p50, p90 y p99).



Tasa de error del servicio, pegada a 0 %.



Traza GET /clients: 14,91 s en total, SELECT de 406 ms.



9. Preguntas de análisis

1. Lo primero que se degradó al cruzar el punto de saturación fue la latencia, no la tasa de error. Entre 10 y 25 usuarios el p95 pasa de 278 ms a 959 ms y el caudal deja de subir. El error del APM sigue en 0 % hasta el final. k6 solo ve fallos del cliente en el escalón de 400 usuarios, cuando la espera ya va en decenas de segundos. Ese orden indica que la API sigue aceptando el trabajo y lo termina en HTTP 200, pero lo encola: responde tarde antes de empezar a fallar.

2. La proporción sí cambió con la carga, y se midió en GET /clients, que es el 70 % del escenario. En el escalón 1 el p50 de la petición es 37 ms y el del SELECT es 22 ms: alrededor del 60 % del tiempo está en la consulta. En el escalón 6 el p50 de la petición es 15.050 ms y el del SELECT es 202 ms: alrededor del 1 %. El tiempo que se agrega al subir usuarios se queda en la aplicación. La traza de las 10:30, con 14,91 s y un SELECT de 406 ms, es un caso de ese tramo.

3. El throughput no siguió creciendo. El máximo es el escalón 1, con 72 peticiones por segundo. Después baja en cada escalón: 47, 38, 31, 27 y 24 peticiones por segundo.

4. La memoria no se agotó: el heap de Node se quedó cerca de 68 MB y el contenedor de la API tiene 512 MB. El tope de conexiones de MySQL es 151 y el SELECT p50 solo pasó de 22 ms a 202 ms, así que la consulta no es la espera de 15 s. No hay una muestra de CPU del contenedor ni de conexiones activas, porque no se corrió docker stats en cada escalón. Con lo que sí quedó, el recurso que impone el límite es la API: un solo proceso Node limitado a 1 CPU. El event loop llegó a utilización 1,0 y, en cuanto los usuarios pasan de 10, el caudal baja y la espera crece dentro de la petición, no dentro del SELECT.

5. El cambio concreto es paginar GET /clients. Esa operación es el 70 % de la carga y devuelve la tabla completa. Durante la prueba los POST fueron agrandando la tabla, y las trazas muestran que el SELECT se mantuvo corto mientras la petición pasó de 37 ms a 15 s. Devolver una página reduce el trabajo por petición en el proceso que ya no acepta más caudal. El siguiente ajuste, si después de paginar el límite sigue siendo la API, es

subir su tope de 1 CPU.

10. Declaración de uso de inteligencia artificial

Se utilizó Cursor, con el modelo Grok, como asistencia durante este taller.

- Montar SigNoz, el colector OpenTelemetry y la instrumentación de la API y del consumidor.
- Escribir el escenario k6 y los límites de CPU y memoria de los contenedores.
- Ejecutar la prueba de unos 20 minutos y leer las trazas en ClickHouse.
- Tomar las capturas de SigNoz en tamaño de escritorio y guardarlas en postman/.
- Redactar este informe, la tabla, las tres gráficas y las respuestas de análisis.

Los números de la tabla no se estimaron: salen de las trazas de la corrida del 26 de octubre de 2026 y del contador de iteraciones de k6. Las celdas de CPU, memoria y conexiones quedaron vacías a propósito, porque esa muestra no se tomó. El estudiante asume la responsabilidad del código entregado, de la medición y de la comprensión de la solución.